

In the Fundamentals of Machine Learning course, your projects require you to write and run Python code. You will implement, modify, or run one or more learning algorithms, train models with multiple datasets, test the accuracy of your models, and produce graphs and tables showing the performance of your models.

You will be given a code shell to start with. For this project, the code shell already implements the four Naïve Bayes variants described below. Your task is to decide which variant applies to each dataset, call the provided functions with the right arguments, run the experiments, and report on the results. Your work is the notebook cells marked **TODO** and the report. You will not need to import any libraries besides those provided. Use the code shell as a guide, but feel free to restructure the code as you prefer. You may discuss algorithms with other students or chatbots, but it is up to you to write the code and the report.

1 Overview

To complete this assignment, you will work with four variants of the Naïve Bayes learning algorithm, which the code shell implements for you. Two of these algorithms will be applied to text datasets, one will apply to categorical features, and one will apply to continuous numeric features. You will test the models on two text datasets and two tabular datasets, and explain in your report why each variant fits the data it is used on.

2 Algorithms

Naïve Bayes classifiers use Maximum A Posteriori estimates $\arg \max_{y=l \in L} p(y=l|x) = \arg \max_{l \in L} p(y=l)p(x|y=l)$ to find the appropriate label for an input instance $x = \langle x_1, \dots, x_n \rangle$ containing n features. The label prediction for x is the label or class with the largest conditional probability, utilizing the Naïve Bayes assumption, as follows:

$$\hat{y} = \arg \max_{l \in L} p(y=l) \prod_{i=1}^n p(x_i|y=l). \quad (1)$$

For numerical stability, maximize the log of the conditional probability instead.

$$\hat{y} = \arg \max_{l \in L} \ln p(y=l) + \sum_{i=1}^n \ln p(x_i|y=l). \quad (2)$$

$p(y=l)$ is estimated by the fraction of training samples with label l . Naïve Bayes variants differ in how they calculate $p(x_i|y=l)$, including what counts as a feature and assumptions about variable (for Multinomial and Multivariate Bernoulli variants) or fixed length (Categorical and Gaussian variants) input instances. The estimates below use a Laplace smoothing constant k , which adds k to every count so that no probability is ever zero. This imposes a uniform prior on the model. Smoothing applies to every variant except Gaussian. Use $k=1$ everywhere in this project.

The code shell implements the following four Naïve Bayes algorithm variants. For each variant, the underlying generative process assumption is that first the label of the input is generated by sampling the distribution $p(y=l)$, $l \in L$.

Multi-Variate Bernoulli Naïve Bayes: In this model, we assume that each input text is generated by selecting whether or not each word in the vocabulary should be included depending on the class label l previously sampled. Let x_w be a binary variable which takes on values **true** or **false** depending on whether word $w \in V$ is included in the input text x or not. One such variable is drawn for each word of the vocabulary. The Multi-Variate Bernoulli¹ probability $p_B(x_w|y = l)$ is estimated $\forall w \in V, l \in L$ as

$$p_B(x_w = \text{true}|y = l) = \frac{N_{instances}(in(w, x), y = l) + k}{N_{instances}(y = l) + 2k}, \quad (3)$$

where $in(w, x)$ returns **true** if the word w is present in the input text x , $N_{instances}(y = l)$ is the total number of instances of class l and $N_{instances}(in(w, x), y = l)$ is the number of instances of class l which contain the word w . Note that $p_B(x_w = \text{false}|y = l) = 1 - p_B(x_w = \text{true}|y = l)$.

For predicting the label of a new example, x , with the Multi-Variate Bernoulli variant of Naïve Bayes, we adapt Equation 2 as

$$\hat{y} = \arg \max_{l \in L} \ln p(y = l) + \sum_{w \in V} \ln p_B(x_w|y = l).$$

In this adapted equation, $\forall w \in V$, use the value $p_B(x_w = \text{true}|y = l)$ when $in(w, x)$ returns true, and $p_B(x_w = \text{false}|y = l)$ otherwise. A word that is absent from x therefore still contributes a term. The absence of a word is evidence in its own right: a word that is common in one class but missing from x pushes the score away from that class. This is what separates this variant from Multinomial Naïve Bayes, which only counts the words that are present.

Summing over the whole vocabulary for every instance would be slow. Instead, for efficiency, the provided code computes $\sum_{w \in V} \ln p_B(x_w = \text{false}|y = l)$ once per class, and then, for each word in x , subtracts that word's $\ln p_B(x_w = \text{false}|y = l)$ and adds $\ln p_B(x_w = \text{true}|y = l)$ in its place.

For the other three Naïve Bayes variants, described below, predicting labels for new instances can directly use Equation 2.

Multinomial Naïve Bayes: The generative process for this model assumes that given the label l and input length n , each word $x_i, \forall i \in \{1, \dots, n\}$ is independently sampled from the same distribution $p_M(x_i = w|y = l) = p_M(w|y = l)$, where $w \in V$ and V is the vocabulary. $p_M(x_i = w|y = l)$, which is the same for any position i , is estimated as

$$p_M(w|y = l) = \frac{N_{words}(w, y = l) + k}{N_{words}(y = l) + |V|k}, \quad (4)$$

where $N_{words}(y = l)$ is the total number of words in all instances of class l and $N_{words}(w, y = l)$ is the number of occurrences of the word w in all instances of class l . Here n is the number of words in the instance, so a word that occurs twice in x contributes two terms to Equation 2. A word of the vocabulary that does not appear in x contributes nothing here, unlike the Multi-Variate Bernoulli variant above.

Categorical Naïve Bayes: In this model, we assume that every instance $x = \langle x_1, \dots, x_n \rangle$, is a *fixed-length sequence of categorical features*. Each $x_i \in C_i$ is drawn from a feature-specific distribution $p_C(x_i = c|y = l)$ estimated $\forall i, c \in C_i, l \in L$ as

$$p_C(x_i = c|y = l) = \frac{N_{instances}(x_i = c, y = l) + k}{N_{instances}(x_i \in C_i, y = l) + k|C_i|}. \quad (5)$$

¹Multi-Variate Bernoulli Naïve Bayes is also called Bernoulli Naïve Bayes. That is the name scikit-learn uses.

C_i is the set of values that x_i takes in the training data. $N_{instances}(x_i \in C_i, y = l)$ is the total number of training instances of class l that are not missing x_i and $N_{instances}(x_i = c, y = l)$ is the number of training instances of class l in which $x_i = c$. A test value that never appears in the training data has $N_{instances}(x_i = c, y = l) = 0$.

Gaussian Naïve Bayes: In this model, we assume that every instance $x = \langle x_1, \dots, x_n \rangle$, is a *fixed-length sequence of numeric, Gaussian distributed features*. Each x_i is drawn from a feature-specific Gaussian distribution $p_G(x_i|y = l)$ with mean $\mu_{i,l}$ and variance $\sigma_{i,l}^2$ estimated $\forall i, l \in L$ as

$$p_G(x_i|y = l) = \frac{1}{\sqrt{2\pi\sigma_{i,l}^2}} \exp\left(-\frac{(x_i - \mu_{i,l})^2}{2\sigma_{i,l}^2}\right). \quad (6)$$

$\mu_{i,l}$ is the mean and $\sigma_{i,l}^2$ is the variance (the mean squared deviation from $\mu_{i,l}$) of the feature x_i in training instances of class l that are not missing x_i . Laplace smoothing does not apply to the Gaussian Naïve Bayes model. A feature that takes the same value in every training instance of a class has $\sigma_{i,l}^2 = 0$, which leaves Equation 6 undefined. To avoid this, add $\epsilon = 0.01$ to every estimated variance.

Combining Categorical and Gaussian Naïve Bayes: When an instance has categorical features $x_i, i \in I_C$, and real-valued features $x_j, j \in I_G$, use Equation 5 for the former and Equation 6 for the latter within a single model:

$$\hat{y} = \arg \max_{l \in L} \ln p(y = l) + \sum_{i \in I_C} \ln p_C(x_i|y = l) + \sum_{j \in I_G} \ln p_G(x_j|y = l). \quad (7)$$

The class probability $p(y = l)$ appears once. A feature whose value is missing from an instance contributes no term to the sum for that instance.

Choose the variant by the form of the input. The Multinomial and Multi-Variate Bernoulli variants are for text, where instances vary in length. The Categorical and Gaussian variants are for fixed-length instances: Categorical for categorical features and Gaussian for real-valued features. When an instance has both kinds of features, combine the two as in Equation 7.

Build the vocabulary V from the training data only. During inference, skip words that are out of vocabulary.

You can read more about the two text variants in Chapter 13 of Manning, Raghavan, and Schütze's *Introduction to Information Retrieval*. Section 13.2 covers the Multinomial variant, Section 13.3 covers the Multi-Variate Bernoulli variant, and Table 13.3 in Section 13.4 compares the two. For Laplace smoothing and the Categorical estimate, read the Smoothing section of Note 19 from UC Berkeley's CS 188.

These readings use different symbols. The book writes terms t and classes c for our words w and labels l , and fixes $k = 1$. The note writes $|X|$ for our $|C_i|$, and its denominator counts every instance of the class, while ours counts only the instances that are not missing x_i .

Comparing with scikit-learn

scikit-learn implements each of the four variants: [BernoulliNB](#), [MultinomialNB](#), [CategoricalNB](#), and [GaussianNB](#). Its [Naïve Bayes User Guide](#) describes them in the same terms as this document. The provided code is the reference for what each variant computes. After running it on a dataset, run the matching scikit-learn class on the same folds and report both accuracies side by side. The code shell provides the cross validation loop for the library models; your job is to choose the right class for each dataset, pass it the smoothing constant (scikit-learn calls `k_alpha`), and read the result. For the Titanic dataset, which needs Equation 7, the code shell combines [CategoricalNB](#) and [GaussianNB](#) for you.

Do not expect the two accuracies to be identical. The library tokenizes, smooths, encodes categories, and handles missing values in its own way, and `GaussianNB` smooths the variance with a fraction of the largest feature variance rather than a fixed ϵ . Where the accuracies differ, say what in the two implementations could explain the difference. Where they agree, say why the two computations are the same.

3 Datasets

There are four datasets you will use to test the models.

3.1 Text Datasets

Use both the Multinomial and the Multi-Variate Bernoulli variants on each of the two text datasets. Convert each text to lowercase and split it into word tokens before training or prediction. The code shell does this for you.

Spam Ham: This Kaggle dataset includes the text of 5,728 emails originating from various legitimate or spam sources, of which 1,368 are spam. The goal of the task is to sort new emails according to whether they are spam or important emails.

Newsgroups: This Scikit Learn dataset includes the text of around 18000 news articles, each discussing one of 20 topics. The goal is to classify news articles according to their topic of interest. Load the articles without their headers, footers, and quoted replies, so that the models learn from the body text alone.

3.2 Fixed Length (Tabular) Datasets

Handwritten Digits: This is the same UCI ML handwritten digits dataset that you used in the previous project. Treat the ordinal pixel values as numeric data and use Gaussian Naïve Bayes.

Titanic: This Kaggle dataset includes information about 891 of the passengers on board the RMS Titanic. It includes categorical and numeric features, and some features are missing. The goal is to predict which passengers survived the deadly crashing of the vessel. Use the features `Pclass`, `Sex`, `Embarked`, `Age`, `SibSp`, `Parch`, and `Fare`. Do not use `PassengerId`, `Name`, or `Ticket`, which identify a passenger rather than describe one, or `Cabin`, which is missing for most passengers. Decide which of the seven features are categorical and which are numeric. Use a combination of Categorical and Gaussian Naïve Bayes variants to handle all features.

4 Analysis and Report

Evaluate every model with 5-fold cross validation. Shuffle the dataset once, split it into five folds of nearly equal size, and for each fold train on the other four folds and test on that fold. The code shell provides this loop. For each dataset and variant, report the mean test accuracy over the five folds and one confusion matrix that adds up the test predictions of all five folds, so that every instance is counted once.

- **30pts** - Train Multinomial Naïve Bayes models with cross validation on Spam/Ham and Newsgroups datasets. Report the accuracy and confusion matrix of each dataset. Run `MultinomialNB` on the same folds, report its accuracy beside yours, and comment on any difference.

- **30pts** - Train Multi-Variate Bernoulli Naïve Bayes models with cross validation on Spam/Ham and Newsgroups datasets. Report the accuracy and confusion matrix for each dataset, and compare to Multinomial results. Explain which variant does better on each dataset and why. Run `BernoulliNB` on the same folds, report its accuracy beside yours, and comment on any difference.
- **20pts** - Train Gaussian Naïve Bayes models on the Handwritten Digits dataset and report on the accuracy and confusion matrix of the models. Identify the digits that are confused most often. Run `GaussianNB` on the same folds, report its accuracy beside yours, and explain any difference.
- **20pts** - Train Gaussian Naïve Bayes models on the numeric features of the Titanic dataset, and Categorical Naïve Bayes models on the categorical features. Use both algorithms together as a single model, producing one likelihood for each class (Equation 7). Make sure that you apply the class probability $p(y = l)$ only once. Missing values contribute no term, as in Equation 7. Report the accuracy and confusion matrix of the models, and state which features you treated as categorical and why. Run the provided scikit-learn combination of `CategoricalNB` and `GaussianNB` on the same folds, report its accuracy beside yours, and explain any difference.

In your report, state for each dataset which variant you used and explain why that variant fits the form of the data. Include a table with one row per dataset and variant that shows the provided implementation's accuracy beside the scikit-learn accuracy.

5 Submission

Notebooks: Submit the completed notebooks with every cell run and its output visible. Cleared notebooks will be graded as though the code did not run.

Report: Submit the report as a single PDF containing the tables, confusion matrices, and comments described above.

Where: Submit both through Harvey.